UTTAR PRADESH RAJARSHI TANDON OPEN UNIVERSITY

Shantipuram (Sector-F), Phaphamau, Prayagraj – 211013

PROJECT TITLE

[Paper Title]

Submitted by

[Student Name]

Enrollment No.: [Enrollment Number]

Under the guidance of

[Guide Name]

[Designation, College Name]

In partial fulfilment of the requirements for the degree of

BACHELOR OF SCIENCE (MATHEMATICS)

Course Code: [Course Code] Type: Research Project

[Month Year]

Certificate

This is to certify that the project report entitled *[Paper Title]* submitted by [Student Name], Enrollment No. [Enrollment Number], in partial fulfilment of the requirements for the degree of Bachelor of Science (Mathematics) of Uttar Pradesh Rajarshi Tandon Open University, is an original work carried out under my supervision.

[Guide Name]
Guide, [College Name]

[Co-ordinator Name]
Co-ordinator, [College Name]

Declaration

I declare that this project is my own work, that every source I have used is acknowledged in the text and in the references, and that no part of it has been submitted for any other degree.

Place: [Place] [Student Name]
Date: [Date] Signature

Acknowledgement

[Write your own acknowledgement: guide, teachers, family, and the open-university support staff.]

Abstract

[Write a 200–250 word abstract after the paper is complete. State the problem, the methods, the main results, and the computational checks.]

Contents

Certificate **1**

Declaration **2**

Acknowledgement **3**

Abstract **4**

1 Introduction **9**

1.1 Motivation 9
1.2 Objectives 9
1.3 Scope and Limitations 10
1.4 Literature Review 10
1.5 Organisation of the Project 10

2 Preliminaries **11**

2.1 Notation 11
2.2 The Well-Ordering Principle and Induction 11
2.3 Division with Remainder 11
2.4 Greatest Common Divisor 12
2.5 Basic Facts about Primes 12
2.6 Summary 13

3 Primes and the Fundamental Theorem of Arithmetic **14**

3.1 Existence of Prime Factorisation 14
3.2 Uniqueness of Prime Factorisation 14
3.3 Infinitude of Primes 15
3.4 Listing Primes: The Sieve of Eratosthenes 15
3.5 Remark on the Distribution of Primes 15
3.6 Summary 16

4 Congruences, Fermat's and Euler's Theorems **17**
4.1 Congruences . . . 17
4.2 Inverses Modulo n . . . 17
4.3 Euler's Totient Function . . . 18
4.4 Reduced Residue Systems . . . 18
4.5 Euler's Theorem . . . 19
4.6 Fermat's Little Theorem . . . 19
4.7 Fast Modular Exponentiation . . . 19
4.8 Summary . . . 19

5 Primitive Roots and Quadratic Residues **20**
5.1 Order of an Element . . . 20
5.2 Primitive Roots . . . 20
5.3 Discrete Logarithms . . . 21
5.4 Quadratic Residues . . . 21
5.5 Summary . . . 22

6 Applications to Public-Key Cryptography **23**
6.1 The RSA Cryptosystem . . . 23
6.1.1 Key generation . . . 23
6.1.2 Encryption and decryption . . . 23
6.1.3 A worked example with small primes . . . 24
6.2 The Diffie–Hellman Key Exchange . . . 24
6.3 Security Assumptions . . . 24
6.4 Summary . . . 25

7 Computational Verification and Results **26**
7.1 Primes up to 100 . . . 26
7.2 Euler's totient . . . 26
7.3 Extended Euclidean algorithm . . . 27
7.4 Modular exponentiation . . . 27
7.5 Limits of the checks . . . 27
7.6 Summary . . . 28

8 Conclusion and Future Work **29**
8.1 Summary of Results . . . 29
8.2 Limitations . . . 29
8.3 Future Work . . . 30
8.4 Closing Remark . . . 30

A Python Code Listings **31**
A.1 Routines 31
A.2 Source of `numtheory.py` 31
A.3 Source of the tests 33

References **36**

List of Tables

7.1 Values of $\varphi(n)$ for $1 \leq n \leq 20$ 26
7.2 Bézout coefficients from `ext_gcd` 27
7.3 Results of `modpow` 27

Chapter 1

Introduction

Number theory is the branch of mathematics that studies the integers and their properties. Its questions are often easy to state and very hard to answer. Why are there infinitely many primes? How are primes distributed among the integers? When is one integer a square modulo another? For more than two thousand years these questions were studied for their own sake, and the subject was once regarded as the purest part of mathematics.

This changed in the twentieth century. The discovery of public-key cryptography in 1976 by Diffie and Hellman [1], and the RSA cryptosystem of Rivest, Shamir and Adleman in 1978 [5], showed that elementary facts about primes and modular arithmetic underpin the security of everyday communication, banking and online commerce.

1.1 Motivation

The aim of this project is to present the elementary theory of integers in a self-contained way, with full proofs, and then to show how these results are used in cryptography. The project is written for a B.Sc student who has completed a first course in algebra and calculus. It follows the classical textbooks of Niven, Zuckerman and Montgomery [4] and Hardy and Wright [2], and it uses Python to verify the examples.

1.2 Objectives

The objectives of the project are:

1. to state and prove the basic results on divisibility, the greatest common divisor, and the fundamental theorem of arithmetic;

2. to develop the theory of congruences, including Fermat's and Euler's theorems;

3. to study primitive roots and their role in the discrete logarithm problem;

4. to explain how RSA and the Diffie–Hellman key exchange use these results;

5. to verify the main algorithms computationally, with code that can be run and tested.

1.3 Scope and Limitations

The project deals only with elementary number theory. Analytic results such as the prime number theorem are mentioned in remarks but are not proved. The cryptographic chapter describes the mathematics of the schemes; it does not treat implementation security, side-channel attacks or standardised parameter sizes, which are areas for further study.

1.4 Literature Review

Euclid's *Elements*, Book VII, contains the first proof that there are infinitely many primes and the algorithm for the greatest common divisor that bears his name. Gauss's *Disquisitiones Arithmeticae* (1801) introduced the systematic notation of congruences used throughout this work. Modern textbooks such as Hardy and Wright [2] and Koblitz [3] give broader treatments. Stinson [6] presents the cryptographic side in detail.

1.5 Organisation of the Project

Chapter 2 collects the preliminary notions. Chapter 3 treats divisibility and primes. Chapter 4 develops congruences and the theorems of Fermat and Euler. Chapter 5 covers primitive roots and quadratic residues. Chapter 6 applies the theory to cryptography. Chapter 7 reports the computational checks, and Chapter 8 concludes. The Python code is listed in the Appendix.

Chapter 2

Preliminaries

This chapter fixes notation and states the facts from algebra and the well-ordering principle that are used later. Throughout, $\mathbb{Z}$ denotes the integers and $\mathbb{N} = \{1, 2, 3, \dots\}$.

2.1 Notation

Definition 2.1. Let $a, b \in \mathbb{Z}$. We write $a \mid b$ (read "a divides b") if there is an integer c with $b = ac$. If $a \nmid b$, then b is not a multiple of a.

Definition 2.2. An integer $p > 1$ is *prime* if its only positive divisors are 1 and p. An integer $n > 1$ that is not prime is *composite*.

2.2 The Well-Ordering Principle and Induction

Definition 2.3 (Well-ordering principle)**.** Every nonempty set of non-negative integers has a least element.

The well-ordering principle is equivalent to the principle of mathematical induction. Induction is used in several proofs below in the following form: if $P(1)$ holds and $P(n) \Rightarrow P(n+1)$ for every $n \in \mathbb{N}$, then $P(n)$ holds for all $n \in \mathbb{N}$.

2.3 Division with Remainder

Theorem 2.4 (Division algorithm)**.** *Let $a, b \in \mathbb{Z}$ with $b > 0$. Then there exist unique integers q and r such that*

$$a = bq + r, \qquad 0 \le r < b.$$

Proof. Consider the set $S = \{a - bk : k \in \mathbb{Z},\ a - bk \ge 0\}$. It is nonempty: taking $k = -|a|$ gives $a + b|a| \ge a + |a| \ge 0$, since $b \ge 1$. By well-ordering, S has a least element

$r = a - bq$. Then $r \geq 0$. If $r \geq b$, then $r - b = a - b(q+1)$ would be a smaller non-negative element of S, a contradiction. Hence $0 \leq r < b$.

For uniqueness, suppose $a = bq + r = bq' + r'$ with $0 \leq r, r' < b$. Then $b(q - q') = r' - r$, so $b \mid (r' - r)$. But $|r' - r| < b$, so $r' - r = 0$, and therefore $r = r'$ and $q = q'$. □

2.4 Greatest Common Divisor

Definition 2.5. For $a, b \in \mathbb{Z}$, not both zero, the *greatest common divisor* $\gcd(a, b)$ is the largest integer that divides both a and b. We set $\gcd(0, 0) = 0$. Integers a, b are *coprime* if $\gcd(a, b) = 1$.

Theorem 2.6 (Euclidean algorithm)**.** *Let $a, b \in \mathbb{N}$ with $b \leq a$. Define $r_0 = a$, $r_1 = b$ and, while $r_k \neq 0$, let r_{k+1} be the remainder of r_{k-1} upon division by r_k. Then the last nonzero remainder equals* $\gcd(a, b)$.

Proof. The key observation is that $\gcd(r_{k-1}, r_k) = \gcd(r_k, r_{k+1})$. Indeed, $r_{k+1} = r_{k-1} - q r_k$ for some integer q, so every common divisor of r_{k-1} and r_k divides r_{k+1}, and conversely every common divisor of r_k and r_{k+1} divides r_{k-1}. The sequence of remainders is strictly decreasing and non-negative, so it reaches 0 after finitely many steps. When $r_{k+1} = 0$, we have $\gcd(r_{k-1}, r_k) = \gcd(r_k, 0) = r_k$, which is the last nonzero remainder. □

Theorem 2.7 (Bézout's identity)**.** *For $a, b \in \mathbb{Z}$, not both zero, there exist integers x, y such that*

$$ax + by = \gcd(a, b).$$

Proof. Let $S = \{ax + by : x, y \in \mathbb{Z},\ ax + by > 0\}$. This set is nonempty because $a^2 + b^2 > 0$ belongs to it. Let $d = ax_0 + by_0$ be its least element. Divide a by d: $a = dq + r$ with $0 \leq r < d$. Then $r = a - dq = a(1 - qx_0) + b(-qy_0)$ is of the form $ax + by$. Minimality of d forces $r = 0$, so $d \mid a$. Similarly $d \mid b$. Any common divisor of a and b divides d, so $d = \gcd(a, b)$. □

Remark 2.8. The coefficients x, y in Bézout's identity can be found by running the Euclidean algorithm backwards. This is the *extended Euclidean algorithm*, which is implemented in the Appendix. It is the basis for computing modular inverses in RSA key generation.

Corollary 2.9. *Integers a, b are coprime if and only if there exist $x, y \in \mathbb{Z}$ with $ax + by = 1$.*

2.5 Basic Facts about Primes

Lemma 2.10 (Euclid's lemma)**.** *Let p be prime and $a, b \in \mathbb{Z}$. If $p \mid ab$, then $p \mid a$ or $p \mid b$.*

Proof. Suppose $p \nmid a$. Then $\gcd(p, a) = 1$, because the only positive divisors of p are 1 and p. By Corollary 2.9 there are integers x, y with $px + ay = 1$. Multiplying by b gives $b = pbx + aby$. Since p divides pbx and ab, it divides b. □

Example 2.11. Take $p = 5$, $a = 6$, $b = 10$. Then $5 \mid 60 = 6 \cdot 10$, and indeed $5 \mid 10$. Take instead $a = 2$, $b = 3$, $p = 6$: here 6 is not prime, and $6 \mid 6 = 2 \cdot 3$ while $6 \nmid 2$ and $6 \nmid 3$. This shows why primality is needed in Lemma 2.10.

2.6 Summary

This chapter introduced divisibility, the division algorithm, the Euclidean algorithm and Bézout's identity, together with Euclid's lemma. These tools are used repeatedly in Chapter 3 to prove the fundamental theorem of arithmetic.

Chapter 3

Primes and the Fundamental Theorem of Arithmetic

Primes are the building blocks of the integers. This chapter proves that every integer greater than 1 factors into primes in exactly one way, that there are infinitely many primes, and how primes can be listed efficiently.

3.1 Existence of Prime Factorisation

Theorem 3.1. *Every integer $n \geq 2$ can be written as a product of primes.*

Proof. We use strong induction on n. If n is prime, there is nothing to prove. Otherwise $n = ab$ with $1 < a, b < n$. By the induction hypothesis, both a and b are products of primes, and so is $n = ab$. □

3.2 Uniqueness of Prime Factorisation

Theorem 3.2 (Fundamental theorem of arithmetic)**.** *Every integer $n \geq 2$ can be written as*

$$n = p_1 p_2 \cdots p_r,$$

where $p_1 \leq p_2 \leq \cdots \leq p_r$ are primes. This representation is unique.

Proof. Existence is Theorem 3.1. For uniqueness, suppose

$$p_1 p_2 \cdots p_r = q_1 q_2 \cdots q_s,$$

with all p_i and q_j prime and both lists sorted. We show by induction on r that the two lists coincide.

Since p_1 divides the right-hand side, p_1 divides some q_j by repeated use of Lemma 2.10. As q_j is prime and $p_1 > 1$, we get $p_1 = q_j$. Cancel p_1 from both sides. If $r = 1$, the left

side is now 1, so the right side must also be 1, which forces $s = 1$. If $r > 1$, the induction hypothesis applied to the shorter products gives $r - 1 = s - 1$ and the same primes in the same order. $\square$

Remark 3.3. Uniqueness can fail in other rings. For example, in $\mathbb{Z}[\sqrt{-5}]$ we have $6 = 2 \cdot 3 = (1 + \sqrt{-5})(1 - \sqrt{-5})$, and the factors are irreducible but not prime. This is why Euclid's lemma is the key property that makes the theorem work over $\mathbb{Z}$.

3.3 Infinitude of Primes

Theorem 3.4 (Euclid)**.** *There are infinitely many primes.*

Proof. Suppose there are only finitely many primes $p_1, \dots, p_k$. Let $N = p_1 p_2 \cdots p_k + 1$. Then $N \geq 2$, so N has a prime divisor p by Theorem 3.1. Since p is one of the p_i, p divides $N - p_1 \cdots p_k = 1$, which is impossible. Hence there are infinitely many primes. $\square$

Example 3.5. For $k = 3$ with $p_1 = 2$, $p_2 = 3$, $p_3 = 5$, the number $N = 2 \cdot 3 \cdot 5 + 1 = 31$ is itself prime. The proof does not claim that N is always new, only that some prime not in the list must divide it.

3.4 Listing Primes: The Sieve of Eratosthenes

To find all primes up to n, write the integers from 2 to n and repeatedly remove the multiples of each prime in turn. The following fact limits the work.

Lemma 3.6. *If n is composite, then n has a prime divisor p with $p \leq \sqrt{n}$.*

Proof. Write $n = ab$ with $1 < a \leq b < n$. Then $a^2 \leq ab = n$, so $a \leq \sqrt{n}$. The smallest prime divisor of a is at most a, hence at most $\sqrt{n}$, and it also divides n. $\square$

Consequently, to sieve up to n it is enough to remove multiples of primes $p \leq \sqrt{n}$. The routine `sieve` in Appendix A implements this idea.

Example 3.7. Up to 30, the primes are $2, 3, 5, 7, 11, 13, 17, 19, 23, 29$. There are 10 of them. The primes up to 100 number 25, and the code in `test_numtheory.py` checks this count.

3.5 Remark on the Distribution of Primes

The prime number theorem states that the number $\pi(x)$ of primes up to x satisfies $\pi(x) \sim x / \log x$ as $x \to \infty$. Its proof uses complex analysis and is not given here; see Hardy and Wright [2]. For the purposes of this project the theorem only explains

why primes of about 100 digits are common enough to find by random search, which is important for RSA key generation in Chapter 6.

3.6 Summary

Every integer greater than one has a unique prime factorisation. There are infinitely many primes, and the sieve lists them up to a given bound with work controlled by Lemma 3.6.

Chapter 4

Congruences, Fermat's and Euler's Theorems

Congruences let us do arithmetic with remainders. They are the language in which RSA and Diffie–Hellman are stated.

4.1 Congruences

Definition 4.1. Let $n \geq 1$. We write $a \equiv b \pmod{n}$ if $n \mid (a - b)$. We say a and b are *congruent modulo* n.

Proposition 4.2. *For fixed $n \geq 1$, congruence modulo n is an equivalence relation on $\mathbb{Z}$. Moreover, if $a \equiv b$ and $c \equiv d \pmod{n}$, then $a + c \equiv b + d$ and $ac \equiv bd \pmod{n}$.*

Proof. Reflexivity, symmetry and transitivity follow directly from the definition, since $n \mid 0$, $n \mid (a - b) \Rightarrow n \mid (b - a)$, and $n \mid (a - b)$, $n \mid (b - c)$ imply $n \mid (a - c)$. For the second claim, write $a - b = nk$ and $c - d = nl$. Then $(a + c) - (b + d) = n(k + l)$ and $ac - bd = a(c - d) + d(a - b) = n(al + dk)$. □

Each integer is congruent to exactly one of $0, 1, \ldots, n - 1$ modulo n, by Theorem 2.4. So the congruence classes modulo n form the set $\mathbb{Z}_n = \{0, 1, \ldots, n - 1\}$ with addition and multiplication taken modulo n.

4.2 Inverses Modulo n

Theorem 4.3. *Let $n \geq 2$. The integer a has an inverse modulo n, meaning some x with $ax \equiv 1 \pmod{n}$, if and only if $\gcd(a, n) = 1$. In that case the inverse is unique modulo n.*

Proof. If $ax \equiv 1 \pmod{n}$, then $ax - 1 = ny$ for some integer y, so $ax - ny = 1$ and $\gcd(a, n) = 1$ by Corollary 2.9. Conversely, if $\gcd(a, n) = 1$, Bézout's identity gives

$ax + ny = 1$, so $ax \equiv 1 \pmod{n}$. For uniqueness, if $ax \equiv ax' \equiv 1$, then $n \mid a(x - x')$. Since $\gcd(a, n) = 1$, Euclid's lemma in its general form gives $n \mid (x - x')$. $\square$

Remark 4.4. The extended Euclidean algorithm (`ext_gcd` in the Appendix) computes x in the proof above. It is the standard way to compute modular inverses, and RSA key generation uses it to find the private exponent.

4.3 Euler's Totient Function

Definition 4.5. Euler's totient function is

$$\varphi(n) = \#\{\, k : 1 \le k \le n,\ \gcd(k, n) = 1 \,\}.$$

Proposition 4.6. *If p is prime and $k \ge 1$, then $\varphi(p) = p - 1$ and $\varphi(p^k) = p^k - p^{k-1}$.*

Proof. The integers $1 \le m \le p$ are coprime to p except for $m = p$, so $\varphi(p) = p - 1$. For p^k, the integers not coprime to p^k are exactly the multiples of p in $[1, p^k]$, of which there are p^{k-1}. Hence $\varphi(p^k) = p^k - p^{k-1}$. $\square$

Proposition 4.7. *If $\gcd(m, n) = 1$, then $\varphi(mn) = \varphi(m)\varphi(n)$.*

Proof sketch. The map $k \mapsto (k \bmod m,\ k \bmod n)$ is a bijection between $\mathbb{Z}_{mn}$ and $\mathbb{Z}_m \times \mathbb{Z}_n$ by the Chinese remainder theorem, and $\gcd(k, mn) = 1$ exactly when $\gcd(k, m) = \gcd(k, n) = 1$. The full proof of the bijection is standard and is given in Niven, Zuckerman and Montgomery [4]. $\square$

Example 4.8. $\varphi(10) = \varphi(2)\varphi(5) = 1 \cdot 4 = 4$. The reduced residues modulo 10 are $1, 3, 7, 9$.

4.4 Reduced Residue Systems

A *reduced residue system* modulo n is a set of $\varphi(n)$ integers, one from each congruence class coprime to n.

Lemma 4.9. *If $\gcd(a, n) = 1$ and $r_1, \dots, r_{\varphi(n)}$ is a reduced residue system modulo n, then $ar_1, \dots, ar_{\varphi(n)}$ is also a reduced residue system modulo n.*

Proof. Each ar_i is coprime to n. If $ar_i \equiv ar_j \pmod{n}$, then multiplying by an inverse of a (Theorem 4.3) gives $r_i \equiv r_j$, so $i = j$. Thus the $\varphi(n)$ numbers ar_i lie in distinct coprime classes, which are exactly $\varphi(n)$ in number. $\square$

4.5 Euler's Theorem

Theorem 4.10 (Euler)**.** *If* $\gcd(a, n) = 1$*, then* $a^{\varphi(n)} \equiv 1 \pmod{n}$.

Proof. Let $r_1, \dots, r_{\varphi(n)}$ be a reduced residue system. By Lemma 4.9, the numbers $ar_1, \dots, ar_{\varphi(n)}$ are a permutation of the r_i modulo n. Multiplying all of them together gives

$$a^{\varphi(n)} \, r_1 r_2 \cdots r_{\varphi(n)} \equiv r_1 r_2 \cdots r_{\varphi(n)} \pmod{n}.$$

The product $P = r_1 \cdots r_{\varphi(n)}$ is coprime to n, so we can cancel it, giving $a^{\varphi(n)} \equiv 1 \pmod{n}$. □

4.6 Fermat's Little Theorem

Corollary 4.11 (Fermat)**.** *If* p *is prime and* $p \nmid a$*, then* $a^{p-1} \equiv 1 \pmod{p}$*. Consequently* $a^p \equiv a \pmod{p}$ *for every integer* a*.*

Proof. Apply Theorem 4.10 with $n = p$ and $\varphi(p) = p - 1$ from Proposition 4.6. If $p \mid a$, both sides of $a^p \equiv a$ are $\equiv 0$. □

Example 4.12. $2^{100} \bmod 101$: since 101 is prime and $2 \not\equiv 0$, Fermat gives $2^{100} \equiv 1 \pmod{101}$. This is verified by the routine `modpow(2, 100, 101)` in Appendix A.

4.7 Fast Modular Exponentiation

Computing $a^e \bmod n$ by multiplying a by itself e times is far too slow when e has hundreds of digits. Write e in binary, $e = \sum_i \varepsilon_i 2^i$. Then

$$a^e = \prod_{i:\ \varepsilon_i = 1} a^{2^i},$$

and each a^{2^i} is obtained by squaring the previous one. Reducing modulo n at each step keeps every intermediate number below n^2. This is the square-and-multiply method implemented in `modpow`. It needs about $\log_2 e$ squarings and at most as many multiplications, which is what makes RSA practical.

4.8 Summary

Congruences form a ring $\mathbb{Z}_n$. Its units are the classes coprime to n, and their number is $\varphi(n)$. Euler's theorem generalises Fermat's little theorem, and both are the key facts behind the correctness of RSA in Chapter 6.

Chapter 5

Primitive Roots and Quadratic Residues

This chapter studies the multiplicative structure of $\mathbb{Z}_n$. Primitive roots give a way to write every unit as a power of one element, which is the basis of the discrete logarithm problem used in Diffie–Hellman.

5.1 Order of an Element

Definition 5.1. Let $\gcd(a, n) = 1$. The *order* of a modulo n is the least $k \geq 1$ with $a^k \equiv 1 \pmod{n}$. We write $\operatorname{ord}_n(a) = k$.

The order exists by Euler's theorem (Theorem 4.10), since $a^{\varphi(n)} \equiv 1$.

Proposition 5.2. *If* $a^k \equiv 1 \pmod{n}$, *then* $\operatorname{ord}_n(a) \mid k$.

Proof. Let $d = \operatorname{ord}_n(a)$ and write $k = qd + r$ with $0 \leq r < d$. Then $a^r = a^k (a^d)^{-q} \equiv 1 \pmod{n}$. Minimality of d forces $r = 0$. □

Corollary 5.3. $\operatorname{ord}_n(a) \mid \varphi(n)$ *for every* a *coprime to* n.

5.2 Primitive Roots

Definition 5.4. An integer g with $\gcd(g, n) = 1$ is a *primitive root modulo* n if $\operatorname{ord}_n(g) = \varphi(n)$. Equivalently, the powers $g^0, g^1, \ldots, g^{\varphi(n)-1}$ run through every unit modulo n exactly once.

Example 5.5. Modulo 7, the element 3 is a primitive root. Its powers are $3, 2, 6, 4, 5, 1$, which contain every nonzero residue exactly once. Modulo 8 there is no primitive root, since the units $\{1, 3, 5, 7\}$ all satisfy $a^2 \equiv 1$, so no element has order $\varphi(8) = 4$.

Theorem 5.6 (Existence of primitive roots modulo primes)**.** *For every prime p there is a primitive root modulo p. Equivalently, the multiplicative group $\mathbb{Z}_p^\times$ is cyclic.*

Proof outline. Let $d \mid p-1$. The polynomial $x^d - 1$ over the field $\mathbb{Z}_p$ has at most d roots, so at most d elements satisfy $a^d = 1$. Counting elements by order gives $\sum_{d \mid p-1} \psi(d) = p-1$, where $\psi(d)$ is the number of elements of order d. If no element had order $p-1$, then each $\psi(d)$ with $d \mid p-1$, $d < p-1$ would be at most $\varphi(d)$, and the sum would fall short of $p-1$. The full argument, including the identity $\sum_{d \mid m} \varphi(d) = m$, is in Hardy and Wright [2]. □

Remark 5.7. Primitive roots exist modulo n exactly when $n = 1, 2, 4, p^k, 2p^k$ with p an odd prime. This is stated without proof and is used only as a remark.

5.3 Discrete Logarithms

If g is a primitive root modulo p and $\gcd(a, p) = 1$, there is a unique exponent x in $\{0, \dots, p-2\}$ with $g^x \equiv a \pmod{p}$. This x is the *discrete logarithm* of a to base g.

Computing $g^x \bmod p$ is fast by Chapter 4. Recovering x from g^x is believed to be hard for large p, and this one-way behaviour is what the Diffie–Hellman scheme in Chapter 6 relies on.

5.4 Quadratic Residues

Definition 5.8. Let p be an odd prime and $\gcd(a, p) = 1$. The integer a is a *quadratic residue* modulo p if $x^2 \equiv a \pmod{p}$ has a solution. Otherwise a is a *quadratic non-residue*.

Definition 5.9 (Legendre symbol)**.** For an odd prime p, define $\left(\frac{a}{p}\right) = 1$ if a is a quadratic residue, -1 if it is a non-residue, and 0 if $p \mid a$.

Theorem 5.10 (Euler's criterion)**.** *For an odd prime p and $p \nmid a$,*

$$\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}.$$

Proof. Let g be a primitive root modulo p (Theorem 5.6) and write $a \equiv g^k$. Then $a^{(p-1)/2} \equiv g^{k(p-1)/2}$, which is 1 when k is even and -1 when k is odd, because $g^{(p-1)/2}$ is a square root of 1 different from 1. Squares modulo p are exactly the even powers of g, so the claim follows. □

Proposition 5.11. $\left(\frac{ab}{p}\right) = \left(\frac{a}{p}\right)\left(\frac{b}{p}\right)$.

Proof. Apply Theorem 5.10 to ab, using $(ab)^{(p-1)/2} = a^{(p-1)/2} b^{(p-1)/2}$. □

Example 5.12. Modulo 11, the quadratic residues are $1, 3, 4, 5, 9$. Check: $5^2 = 25 \equiv 3$, and $\left(\frac{5}{11}\right) \equiv 5^5 = 3125 \equiv 1 \pmod{11}$. Thus 5 is a residue, consistent with $5^2 \equiv 3$.

5.5 Summary

Primitive roots make the unit group of $\mathbb{Z}_p$ cyclic, so every unit is a power of one generator. Euler's criterion decides quadratic residuosity by a single modular exponentiation. Both facts reappear in Chapter 6.

Chapter 6

Applications to Public-Key Cryptography

Public-key cryptography lets two parties communicate securely without sharing a secret in advance. This chapter explains the RSA cryptosystem and the Diffie–Hellman key exchange, and shows how the results of the previous chapters make them work.

6.1 The RSA Cryptosystem

6.1.1 Key generation

1. Choose two distinct large primes p and q. Let $n = pq$ and $\varphi(n) = (p-1)(q-1)$, by Proposition 4.7.

2. Choose e with $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$.

3. Compute d with $ed \equiv 1 \pmod{\varphi(n)}$. This uses the extended Euclidean algorithm (Theorem 4.3).

4. The public key is (n, e). The private key is d, and $p, q, \varphi(n)$ must be kept secret.

6.1.2 Encryption and decryption

A message is an integer m with $0 \le m < n$. Encryption is $c \equiv m^e \pmod{n}$, and decryption is $m \equiv c^d \pmod{n}$.

Theorem 6.1 (Correctness of RSA)**.** *If $ed \equiv 1 \pmod{\varphi(n)}$, then $(m^e)^d \equiv m \pmod{n}$ for every integer m.*

Proof. Write $ed = 1 + k\varphi(n)$ for some integer $k \ge 0$. We show $m^{ed} \equiv m \pmod{p}$. If $p \mid m$, both sides are $\equiv 0$. Otherwise Fermat's little theorem (Corollary 4.11) gives $m^{p-1} \equiv 1$,

and since $(p-1) \mid \varphi(n)$,

$$m^{ed} = m \cdot (m^{p-1})^{k(q-1)} \equiv m \pmod{p}.$$

The same argument works modulo q. Since p and q are distinct primes, $m^{ed} \equiv m \pmod{pq}$. □

Remark 6.2. The proof uses Fermat's theorem prime by prime, which is simpler than Euler's theorem modulo n directly. Both give the same result because $\varphi(n)$ is a multiple of $p-1$ and of $q-1$.

6.1.3 A worked example with small primes

Take $p = 61$ and $q = 53$. Then $n = 3233$ and $\varphi(n) = 60 \cdot 52 = 3120$. Choose $e = 17$. The extended Euclidean algorithm gives $d = 2753$, since $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$. To encrypt $m = 65$:

$$c \equiv 65^{17} \equiv 2790 \pmod{3233}.$$

Decryption gives $2790^{2753} \equiv 65 \pmod{3233}$.

These numbers are far too small to be secure. They are used only to check the algebra. The code in `code/` can reproduce them.

6.2 The Diffie–Hellman Key Exchange

Let p be a large prime and g a primitive root modulo p. These public values are shared openly.

1. Alice chooses a secret a and sends $A \equiv g^a \pmod{p}$.
2. Bob chooses a secret b and sends $B \equiv g^b \pmod{p}$.
3. Alice computes $s \equiv B^a \pmod{p}$. Bob computes $s' \equiv A^b \pmod{p}$.

Both computations give the same value, because $B^a \equiv g^{ab} \equiv A^b \pmod{p}$. An eavesdropper who sees p, g, A and B must solve a discrete logarithm problem to find a or b, as discussed in Chapter 5. The original scheme is described in Diffie and Hellman [1].

6.3 Security Assumptions

The security of RSA rests on the belief that factoring $n = pq$ is hard when p and q are large and randomly chosen. Knowing $\varphi(n)$ together with n is enough to recover p and q, so computing $\varphi(n)$ is as hard as factoring. The security of Diffie–Hellman rests

on the discrete logarithm problem. Neither assumption has been proved, and both may fail against future algorithms or quantum computers. Stinson [6] and Koblitz [3] discuss these assumptions and the parameter sizes used in practice.

The mathematics in this chapter does not cover implementation issues such as padding schemes, side-channel attacks, or random number generation. A secure system must address these as well.

6.4 Summary

RSA encrypts by raising to the power e modulo n, and decrypts with the power d. Its correctness follows from Fermat's theorem, and its key generation uses the extended Euclidean algorithm. Diffie–Hellman uses the commutativity of exponentiation together with the difficulty of discrete logarithms.

Chapter 7

Computational Verification and Results

The theorems in Chapters 3–6 are proved by hand. This chapter reports numerical checks of the same statements, computed with the routines in Appendix A. The checks do not replace the proofs. They confirm that the statements and the algorithms agree on concrete inputs, and they catch arithmetic slips in the examples.

All values below were produced by running the code in `research-paper/code/`. The test suite (`python -m pytest research-paper/code`) passes with 312 test cases.

7.1 Primes up to 100

The sieve of Eratosthenes (Section 3.4) returns exactly 25 primes up to 100, in agreement with the count $\pi(100) = 25$:

$$\begin{array}{cccccccccc} 2 & 3 & 5 & 7 & 11 & 13 & 17 & 19 & 23 & 29 \\ 31 & 37 & 41 & 43 & 47 & 53 & 59 & 61 & 67 & 71 \\ 73 & 79 & 83 & 89 & 97 & & & & & \end{array}$$

7.2 Euler's totient

Table 7.1 lists $\varphi(n)$ for $1 \le n \le 20$, computed by `phi`. Each entry agrees with the brute-force count in the test suite, which checks all $n < 300$.

Table 7.1: Values of $\varphi(n)$ for $1 \le n \le 20$

n	1	2	3	4	5	6	7	8	9	10
$\varphi(n)$	1	1	2	2	4	2	6	4	6	4
n	11	12	13	14	15	16	17	18	19	20
$\varphi(n)$	10	4	12	6	8	8	16	6	18	8

A consistency check from Chapter 4: $\sum_{d \mid 12} \varphi(d) = 12$. The code gives 12, as the identity requires.

7.3 Extended Euclidean algorithm

Table 7.2 gives Bézout coefficients for several pairs. In every row, $ax + by = \gcd(a, b)$ holds exactly, which is checked by the code and also by hand.

Table 7.2: Bézout coefficients from `ext_gcd`

a	b	$\gcd(a,b)$	x	y	check $ax+by$
252	198	18	4	-5	$1008 - 990 = 18$
973	301	7	13	-42	$12649 - 12642 = 7$
1071	462	21	-3	7	$-3213 + 3234 = 21$
240	46	2	-9	47	$-2160 + 2162 = 2$
17	3120	1	-367	2	$-6239 + 6240 = 1$

The last row is the one used in the RSA example of Chapter 6: it shows that 17 is invertible modulo $\varphi(3233) = 3120$, with inverse $-367 \equiv 2753 \pmod{3120}$.

7.4 Modular exponentiation

Table 7.3 shows values of `modpow`. The row $2^{100} \bmod 101$ confirms Fermat's little theorem (Corollary 4.11), since 101 is prime. The rows with $3^{200} \bmod 1000003$ and the RSA numbers illustrate the routine on larger exponents.

Table 7.3: Results of `modpow`

base	exponent	modulus	result
2	10	1000	24
7	128	13	3
2	100	101	1
65	17	3233	2790
2790	2753	3233	65
3	200	1000003	333986

Two of these values were also checked by hand. $7^{128} \bmod 13$: since $7^{12} \equiv 1$ and $128 \equiv 8 \pmod{12}$, we get $7^{8} \equiv 3$. The RSA rows reproduce the worked example of Section 6.1.3, where decryption returns the original message 65.

7.5 Limits of the checks

These tests cover small inputs only. The RSA parameters ($n = 3233$) are far too small for real use, and the code does no timing analysis at large sizes. Results for cryptographic sizes would need a separate study, which is noted as future work in Chapter 8.

7.6 Summary

The routines agree with the theorems on every test case. The worked numerical examples in Chapters 4–6 are reproduced exactly.

Chapter 8

Conclusion and Future Work

This project set out to present the elementary theory of integers with full proofs and to show how that theory supports public-key cryptography. This chapter summarises the results, states the limits of the work, and lists directions for further study.

8.1 Summary of Results

1. The division algorithm, the Euclidean algorithm and Bézout's identity (Chapter 2).
2. The existence and uniqueness of prime factorisation, and Euclid's proof that there are infinitely many primes (Chapter 3).
3. Congruence arithmetic, the totient function, and the theorems of Euler and Fermat (Chapter 4).
4. Primitive roots, discrete logarithms and Euler's criterion for quadratic residues (Chapter 5).
5. The correctness proof of RSA and the Diffie–Hellman key exchange (Chapter 6).
6. Computational checks of these statements, with all test cases passing (Chapter 7).

8.2 Limitations

The project is limited in three ways. First, the proofs in Chapters 3 to 5 are given at the level of a B.Sc course, and some results, such as the existence of primitive roots, are only outlined. Second, the cryptographic chapter describes the mathematics of the schemes but not their implementation or the attacks that target implementations. Third, the computations use small numbers. They verify the algebra but say nothing about the running time at cryptographic sizes.

8.3 Future Work

The following topics would extend this project:

- A full proof of the existence of primitive roots modulo p, and of the classification of moduli that have them.
- The quadratic reciprocity law and its use in deciding whether a given integer is a square modulo a prime.
- The prime number theorem, and the Miller–Rabin primality test, which is the probabilistic test used in practice to generate RSA keys.
- Elliptic-curve cryptography, which replaces the multiplicative group modulo p used here with the group of points on an elliptic curve.
- The computational complexity of integer factorisation and discrete logarithms, and a timing study of the routines in Appendix A at realistic key sizes.

8.4 Closing Remark

Number theory was once studied only for its own sake. The results in this project show that the same elementary facts about divisibility and congruences now protect everyday communication. The work also shows that a short computer program can check a theorem's examples, and that checking an example by hand and by machine is a good habit for any mathematician.

Appendix A

Python Code Listings

The routines below are in `research-paper/code/numtheory.py`. Their tests are in `research-paper/code/test_numtheory.py`. Run the tests with `python -m pytest research-paper/code`.

A.1 Routines

- `sieve(n)`: the primes up to n (Eratosthenes).
- `gcd(a, b)`: the Euclidean algorithm (Theorem 2.6).
- `ext_gcd(a, b)`: returns (d, x, y) with $ax + by = d = \gcd(a, b)$ (Theorem 2.7).
- `phi(n)`: Euler's totient, by factoring out primes.
- `modpow(b, e, m)`: square-and-multiply modular exponentiation.

A.2 Source of `numtheory.py`

```
"""Elementary number theory routines used in the research paper.

Each function corresponds to a result in the paper; see the chapter
references in the docstrings.
"""


def sieve(n: int) -> list[int]:
    """Return all primes less than or equal to n (Sieve of Eratosthenes)."""
    if n < 2:
        return []
```

```
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] = False
    p = 2
    while p * p <= n:
        if is_prime[p]:
            for multiple in range(p * p, n + 1, p):
                is_prime[multiple] = False
        p += 1
    return [i for i, flag in enumerate(is_prime) if flag]


def gcd(a: int, b: int) -> int:
    """Greatest common divisor by the Euclidean algorithm (Theorem 2.3)."""
    a, b = abs(a), abs(b)
    while b:
        a, b = b, a % b
    return a


def ext_gcd(a: int, b: int) -> tuple[int, int, int]:
    """Return (d, x, y) with a*x + b*y = d = gcd(a, b) (Theorem 2.4)."""
    old_r, r = a, b
    old_x, x = 1, 0
    old_y, y = 0, 1
    while r:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_x, x = x, old_x - q * x
        old_y, y = y, old_y - q * y
    if old_r < 0:
        return -old_r, -old_x, -old_y
    return old_r, old_x, old_y


def phi(n: int) -> int:
    """Euler's totient: count of 1 <= k <= n with gcd(k, n) = 1 (Chapter 4)."""
    if n < 1:
        raise ValueError("n must be positive")
    result = n
```

```
    m = n
    p = 2
    while p * p <= m:
        if m % p == 0:
            while m % p == 0:
                m //= p
            result -= result // p
        p += 1
    if m > 1:
        result -= result // m
    return result


def modpow(base: int, exp: int, mod: int) -> int:
    """Compute base**exp mod mod by repeated squaring (used in Chapter 4)."""
    if mod <= 0:
        raise ValueError("modulus must be positive")
    if exp < 0:
        raise ValueError("exponent must be non-negative")
    result = 1 % mod
    base %= mod
    while exp:
        if exp & 1:
            result = (result * base) % mod
        base = (base * base) % mod
        exp >>= 1
    return result
```

A.3 Source of the tests

```
import math
import random

import pytest

from numtheory import ext_gcd, gcd, modpow, phi, sieve


def test_sieve_small_primes():
```

```
    assert sieve(30) == [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]


def test_sieve_edge_cases():
    assert sieve(1) == []
    assert sieve(2) == [2]


def test_sieve_count_below_100():
    assert len(sieve(100)) == 25


def test_gcd_matches_math_gcd_random():
    rng = random.Random(0)
    for _ in range(500):
        a = rng.randint(-10**6, 10**6)
        b = rng.randint(-10**6, 10**6)
        assert gcd(a, b) == math.gcd(a, b)


def test_ext_gcd_bezout_identity_random():
    rng = random.Random(1)
    for _ in range(500):
        a = rng.randint(-10**6, 10**6)
        b = rng.randint(-10**6, 10**6)
        d, x, y = ext_gcd(a, b)
        assert d == math.gcd(a, b)
        assert a * x + b * y == d


@pytest.mark.parametrize(
    "base, exp, mod",
    [(2, 10, 1000), (7, 128, 13), (3, 0, 7), (10, 20, 1), (123, 456, 789)],
)
def test_modpow_matches_builtin(base, exp, mod):
    assert modpow(base, exp, mod) == pow(base, exp, mod)


def test_modpow_rejects_bad_input():
```

```
    with pytest.raises(ValueError):
        modpow(2, 3, 0)
    with pytest.raises(ValueError):
        modpow(2, -1, 5)


def brute_phi(n):
    return sum(1 for k in range(1, n + 1) if math.gcd(k, n) == 1)


@pytest.mark.parametrize("n", range(1, 300))
def test_phi_matches_brute_force(n):
    assert phi(n) == brute_phi(n)


def test_phi_known_values():
    assert phi(10) == 4
    assert phi(101) == 100
    assert phi(36) == 12


def test_euler_theorem_numerically():
    rng = random.Random(2)
    for _ in range(200):
        n = rng.randint(2, 5000)
        a = rng.randint(1, 10**6)
        if math.gcd(a, n) == 1:
            assert modpow(a, phi(n), n) == 1
```

References

[1] Whitfield Diffie and Martin E. Hellman. "New Directions in Cryptography". In: *IEEE Transactions on Information Theory* 22.6 (1976), pp. 644–654.

[2] G. H. Hardy and E. M. Wright. *An Introduction to the Theory of Numbers*. 6th. Revised by D. R. Heath-Brown and J. H. Silverman. Oxford University Press, 2008.

[3] Neal Koblitz. *A Course in Number Theory and Cryptography*. 2nd. Springer, 1994.

[4] Ivan Niven, Herbert S. Zuckerman, and Hugh L. Montgomery. *An Introduction to the Theory of Numbers*. 5th. Wiley, 1991.

[5] Ronald L. Rivest, Adi Shamir, and Leonard Adleman. "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems". In: *Communications of the ACM* 21.2 (1978), pp. 120–126.

[6] Douglas R. Stinson. *Cryptography: Theory and Practice*. 3rd. Chapman and Hall/CRC, 2006.